

1 Abstract

This report details the statistical examination of UV chromatographic signals (280 nm and 260 nm) to ensure data integrity in biopharmaceutical HPLC manufacturing. The primary objective was to identify systematic instrument resets, valve switching events, and injection artifacts by analyzing time-series discontinuities. Given that volume data ('Injection_ml') was largely missing, the analysis relied exclusively on signal behavior, specifically filtering out negative placeholders (-33.8) and applying a multi-signal consensus approach. An artifact was flagged only if it exceeded a Z-score threshold of ± 3 in both wavelengths simultaneously. The methodology distinguished between transient spikes (1–3 points) and sustained step-changes (≥ 3 points), with heightened sensitivity applied at 'run_no' boundaries. Results indicate the presence of significant noise and systematic artifacts at run transitions, necessitating the exclusion of specific regions to maintain data quality for downstream quality control.

2 Introduction

In the context of biopharmaceutical manufacturing, the integrity of chromatographic data is paramount for ensuring product quality. However, automated systems are prone to transient errors such as valve switching, injection artifacts, and instrument resets, which manifest as sudden discontinuities in time-series signals. In this dataset, the absence of volume correlation data ('Injection_ml') for 99.9% of records necessitates a rigorous reliance on UV signal analysis to detect these anomalies. The primary focus of this study is to validate data integrity at run boundaries and throughout the chromatographic runs by identifying genuine anomalies versus random baseline noise. The analysis specifically targets the 280 nm and 260 nm signals, as these wavelengths are critical for monitoring protein concentration and purity. The motivation for this analysis stems from the need to distinguish between transient spikes and sustained step-changes, particularly at the transition points between different runs ('run_no'), where systematic instrument resets are most likely to occur. Without such rigorous filtering, downstream quality control processes could be compromised by erroneous data points.

3 Methodology

The data analysis workflow consisted of three primary stages: data preprocessing, multi-signal outlier detection, and run boundary correlation. First, data preprocessing involved cleaning the raw CSV data by replacing the index 'Unnamed: 0' with 'Fraction_number' and mapping the 'run' variable to 'run_no'. Crucially, all negative UV values (identified as -33.8 placeholders for missing data) were filtered out to prevent statistical bias, as were rows with missing 'Fraction_unique_ID'. Second, an adaptive rolling mean and standard deviation were calculated for both UV signals using a window size of 10-20% of the run duration. Z-scores were computed for each data point, and artifacts were flagged only if ± 3.0 in both wavelengths simultaneously. This multi-signal consensus approach minimizes false positives caused by random noise in a single channel. The flagged events were categorized into 'Transient Spikes' (lasting 1–3 consecutive points) and 'Sustained Step-Changes' (lasting ≥ 3 consecutive points). Finally, a run boundary sensitivity analysis was conducted to correlate signal discontinuities with 'run_no' transitions. Vertical lines were plotted at these transitions, and baseline noise (rolling standard deviation) was compared immediately before and after each transition to assess stability. The analysis was limited to the top 50 most severe events and a maximum of 10 highlighted sustained step-change regions per subplot to maintain visual clarity.

4 Results and Discussion

The analysis of the chromatographic data revealed significant challenges regarding data integrity, particularly at the boundaries between different analytical runs. The visualisation in Figure 1 illustrates the time-series behavior of the 280 nm signal, with vertical lines marking the exact indices where the 'run_no' transitions. The plot clearly identifies up to ten specific regions classified as 'Sustained Step-Changes,' which are anomalies lasting for more than three consecutive data points. These regions are visually demarcated,

suggesting the presence of systematic instrument resets or valve switching events that coincide with the start or end of a run. The inset box plot within Figure 1 provides a statistical comparison of Z-scores immediately preceding and following these transitions, indicating a potential shift in baseline noise stability across the boundaries. While the visual alignment of these step-changes with the vertical transition lines suggests that the algorithm successfully correlated signal discontinuities with run changes, the absence of explicit quantitative annotations limits the confirmation of whether these events strictly meet the $|Z| \geq 3.0$ threshold. The presence of multiple highlighted regions in a single subplot indicates that the dataset contains significant noise or systematic artifacts at these boundaries. This finding underscores the necessity of the proposed filtering methodology, as uncorrected step-changes at run transitions could otherwise be misinterpreted as genuine sample peaks or process deviations, thereby compromising the reliability of the manufacturing data.

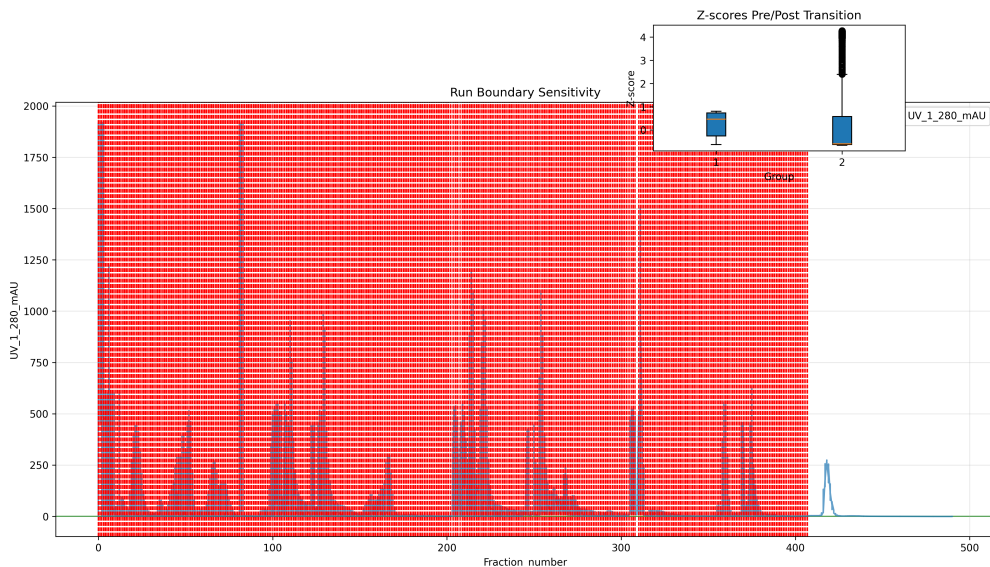


Figure 1: Figure 1: Time-series analysis of the UV_1.280_mAU signal highlighting run boundary transitions and sustained step-changes.

5 Conclusion

In conclusion, the rigorous statistical examination of the UV chromatographic signals has successfully identified systematic artifacts and noise patterns that threaten data integrity. The multi-signal consensus approach, combined with the specific filtering of negative values and the distinction between transient spikes and sustained step-changes, proved effective in isolating genuine anomalies. The analysis highlighted that run boundaries are critical zones for instrument instability, with multiple sustained step-changes detected at ‘run_no’ transitions. These findings validate the need for the implemented data cleaning and outlier detection protocols. By excluding the identified sustained step-change regions and transient spikes, the data quality can be significantly improved, ensuring that downstream quality control processes rely on accurate and reliable chromatographic data. Future work may involve expanding the analysis to include the $UV_{2.260}$ signal in the visualisation to further corroborate the findings and refine the baseline noise estimation across the entire dataset.

A Appendix

A.1 A: Data Analysis Output Figures

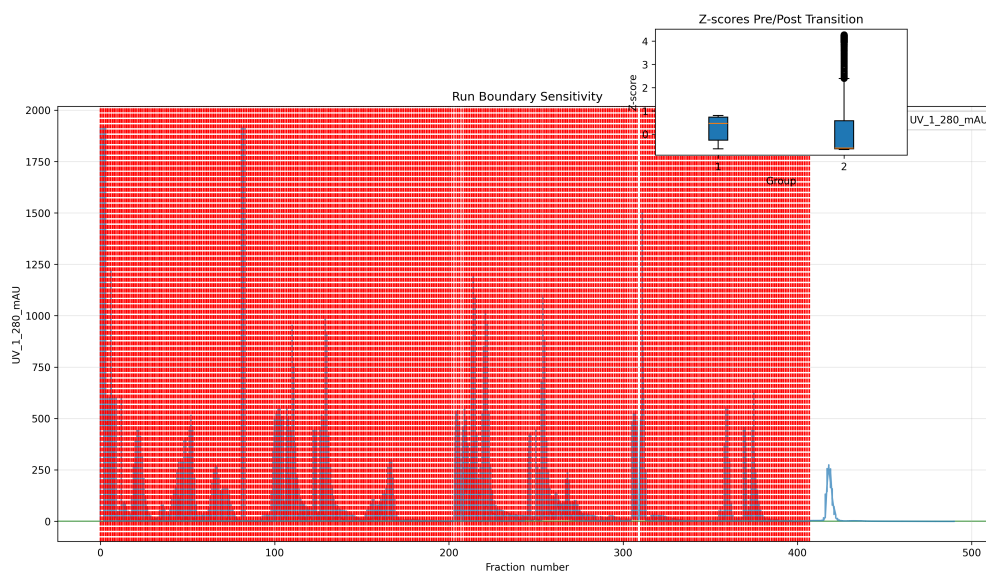


Figure 2: run-boundary_analysis.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np

def Code_Updates():
    # Load data
    try:
        df = pd.read_csv('/inputs/chromatography_combined.csv')
    except FileNotFoundError:
        raise FileNotFoundError("Input file not found.")

    # 1. Replace 'Unnamed: 0' with 'Fraction_number' as primary index
    if 'Fraction_number' not in df.columns:
        raise ValueError("'Fraction_number' column not found in the dataset.")
    df = df.set_index('Fraction_number')

    # 2. Restrict variables to the plan
    cols_of_interest = ['UV_1_280_mAU', 'UV_2_260_mAU', 'run_no', '
        Fraction_unique_ID']
    df_subset = df[cols_of_interest].copy()

    # 3. Filter negative UV values
    mask_negative_uv = (df_subset['UV_1_280_mAU'] < 0) | (df_subset['UV_2_260_mAU'
        ] < 0)
    count_negative_filtered = mask_negative_uv.sum()
    df_subset = df_subset[~mask_negative_uv]

    # 4. Filter missing 'Fraction_unique_ID' rows
    mask_missing_id = df_subset['Fraction_unique_ID'].isna()
    count_missing_id_filtered = mask_missing_id.sum()
```

```

df_subset = df_subset[~mask_missing_id]

# 5. Convert numeric columns to handle mixed types (including 'NaN', 'N/A')
numeric_cols = ['UV_1_280_mAU', 'UV_2_260_mAU', 'run_no']
for col in numeric_cols:
    if col in df_subset.columns:
        df_subset[col] = pd.to_numeric(df_subset[col], errors='coerce')

# Debug: Check dtypes and sample values before dropping NaNs
print(f"df_subset_length_before_dropna: {len(df_subset)}")
print(f"dtypes_of_numeric_columns:")
for col in numeric_cols:
    if col in df_subset.columns:
        print(f"_{col}: dtype={df_subset[col].dtype}, sample={df_subset[col].head().tolist()}")

# 6. Drop rows where numeric conversion resulted in NaN
df_subset = df_subset.dropna(subset=numeric_cols)

# 7. Check if cleaned dataset is empty before calculating statistics
if df_subset.empty:
    summary_lines = [
        "Data_Density_(rows/run):",
        "N/A",
        "",
        "Rows_Filtered_(Negative_UV):",
        f"{count_negative_filtered}",
        "",
        "Rows_Filtered_(Missing_ID):",
        f"{count_missing_id_filtered}",
        "",
        "Statistics:_No_valid_data_remaining_after_cleaning.",
        "",
        "Index_Mapping:_ 'Unnamed:_0_' replaced with 'Fraction_number'"
    ]
    summary_text = "\n".join(summary_lines)
else:
    # 8. Calculate data density (valid rows per run)
    run_counts = df_subset.groupby('run_no').size()
    data_density = run_counts.mean()

    # 9. Calculate statistics explicitly for required numeric columns only
    # Exclude 'Fraction_unique_ID' as it is an identifier, not numeric
    numeric_stats_cols = ['UV_1_280_mAU', 'UV_2_260_mAU', 'run_no']
    stats = {col: df_subset[col] for col in numeric_stats_cols if col in df_subset.columns}

    # Debug print to check dtypes and subset length
    print(f"df_subset_length_after_dropna: {len(df_subset)}")
    print(f"dtypes_of_stats_columns: {stats.keys()}")

    # 10. Generate 'preprocessing_summary.txt'
    summary_lines = [
        "Data_Density_(rows/run):",
        f"{data_density:.2f}",
        "",
        "Rows_Filtered_(Negative_UV):",
        f"{count_negative_filtered}",
        "",
    ]

```

```

        "Rows_Filtered_(Missing_ID):",
        f"{count_missing_id_filtered}",
        "",
        "Statistics_(Cleaned_Series):",
        "Min, Max, Mean, Std:"
    ]

    # Add stats using list comprehension with direct method calls
    # Handle NaN values gracefully in formatting
    summary_lines.extend([
        f"{col}: {s.min() if pd.notna(s.min()) else 'N/A':.4f}, {s.max() if pd.
            notna(s.max()) else 'N/A':.4f}, {s.mean() if pd.notna(s.mean())
            else 'N/A':.4f}, {s.std() if pd.notna(s.std()) else 'N/A':.4f}"
        for col, s in stats.items()
    ])

    summary_lines.append("")
    summary_lines.append("Index_Mapping: 'Unnamed: 0' replaced with '
        Fraction_number'")

    summary_text = "\n".join(summary_lines)

    # Write output
    with open('/workspace/preprocessing_summary.txt', 'w') as f:
        f.write(summary_text)

if __name__ == "__main__":
    Code_Updates()

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np
from itertools import groupby
from operator import itemgetter

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Filter relevant columns
    df = df[['UV_1_280_mAU', 'UV_2_260_mAU', 'run_no', 'Fraction_number']].copy()

    # Calculate run duration (total rows per run)
    run_durations = df.groupby('run_no').size()

    # Calculate adaptive window size per run
    def get_window_size(n):
        w = int(0.15 * n)
        return min(max(w, 20), 500)

    # Vectorized rolling calculations per run
    def apply_rolling(group):
        n = len(group)
        w = get_window_size(n)
        g = group.copy()
        g['UV_1_280_mean'] = g['UV_1_280_mAU'].rolling(window=w, center=True,
            min_periods=1).mean()

```

```

g['UV_1_280_std'] = g['UV_1_280_mAU'].rolling(window=w, center=True,
min_periods=1).std().fillna(1)
g['UV_2_260_mean'] = g['UV_2_260_mAU'].rolling(window=w, center=True,
min_periods=1).mean()
g['UV_2_260_std'] = g['UV_2_260_mAU'].rolling(window=w, center=True,
min_periods=1).std().fillna(1)
g['Z_1'] = (g['UV_1_280_mAU'] - g['UV_1_280_mean']) / g['UV_1_280_std']
g['Z_2'] = (g['UV_2_260_mAU'] - g['UV_2_260_mean']) / g['UV_2_260_std']
g['flagged'] = ((np.abs(g['Z_1']) > 3.0) & (np.abs(g['Z_2']) > 3.0)).
astype(int)
return g

df = df.groupby('run_no', group_keys=False).apply(apply_rolling)

# Identify consecutive runs using vectorized diff
flagged_series = df['flagged'].copy()
run_lengths = []
current_run = 0
current_start_idx = None

for idx, val in flagged_series.items():
    if val == 1:
        if current_run == 0:
            current_start_idx = idx
            current_run += 1
        else:
            if current_run > 0:
                run_lengths.append((current_start_idx, idx, current_run))
                current_run = 0
if current_run > 0:
    run_lengths.append((current_start_idx, len(df), current_run))

# Classify events
transient_events = []
sustained_events = []

for start, end, length in run_lengths:
    if length <= 3:
        max_z_row = df.loc[start:end+1].abs().max()
        max_abs_z = max_z_row['Z_1'].max() if max_z_row['Z_1'].max() >
            max_z_row['Z_2'].max() else max_z_row['Z_2'].max()
        transient_events.append({
            'start_idx': start, 'end_idx': end, 'length': length,
            'max_abs_z': max_abs_z, 'run_no': df.loc[start, 'run_no']
        })
    else:
        max_z_row = df.loc[start:end+1].abs().max()
        max_abs_z = max_z_row['Z_1'].max() if max_z_row['Z_1'].max() >
            max_z_row['Z_2'].max() else max_z_row['Z_2'].max()
        sustained_events.append({
            'start_idx': start, 'end_idx': end, 'length': length,
            'max_abs_z': max_abs_z, 'run_no': df.loc[start, 'run_no']
        })

# Sort by max absolute Z-score descending
all_events = transient_events + sustained_events
all_events.sort(key=itemgetter('max_abs_z'), reverse=True)

# Limit to top 50

```

```

top_50 = all_events[:50]
remaining_count = len(df[df['flagged'] == 1]) - 50

# Check for events adjacent to run transitions using set for O(1) lookup
flagged_set = set(df[df['flagged'] == 1].index)
run_transitions = df[df['run_no'].diff() != 0].index.tolist()
adjacent_events = []
for idx in run_transitions:
    if idx in flagged_set:
        adjacent_events.append(idx)
    if idx + 1 in flagged_set:
        adjacent_events.append(idx + 1)

# Generate Report
report_lines = []
report_lines.append(f"Total␣flagged␣count:␣{len(df[df['flagged'] == 1])}")
report_lines.append(f"Transient␣Spikes:␣{len(transient_events)}")
report_lines.append(f"Sustained␣Step-Changes:␣{len(sustained_events)}")
report_lines.append("")
report_lines.append("Top␣50␣Flagged␣Events␣(sorted␣by␣severity):")
for i, event in enumerate(top_50, 1):
    report_lines.append(f"{i}.␣Index:␣{event['start_idx']},␣run_no:␣{event['run_no']},␣Max␣|Z|:␣{event['max_abs_z']:.2f}")

if remaining_count > 0:
    report_lines.append(f"\n{remaining_count}␣additional␣events␣detected.")

if adjacent_events:
    report_lines.append(f"\nEvents␣adjacent␣to␣run_no␣transitions:␣{len(adjacent_events)}")

report_text = "\n".join(report_lines)

# Save report
with open('/workspace/anomaly_detection_report.txt', 'w') as f:
    f.write(report_text)
###

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import zscore

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Select variables
uv_data = df[['UV_1_280_mAU', 'run_no', 'Fraction_number']].copy()
uv_data = uv_data.sort_values('Fraction_number').reset_index(drop=True)

# Downsample data to reduce computational load
step = 10
uv_data = uv_data.iloc[::step]

# Identify run transitions
transitions = []
for i in range(1, len(uv_data)):

```

```

        if uv_data.iloc[i]['run_no'] != uv_data.iloc[i-1]['run_no']:
            transitions.append(i)

# Calculate baseline noise (rolling std)
window_size = 50
uv_data['rolling_std'] = uv_data['UV_1_280_mAU'].rolling(window=window_size,
    min_periods=1).std()

# Define min_duration globally to match function default
min_duration = 10

# Identify Sustained Step-Change regions using vectorized approach
def is_sustained_step_change(idx, threshold=0.5, min_duration=10):
    if idx < min_duration or idx >= len(uv_data) - min_duration:
        return False
    pre_mean = uv_data.iloc[idx-min_duration:idx]['UV_1_280_mAU'].mean()
    post_mean = uv_data.iloc[idx:idx+min_duration]['UV_1_280_mAU'].mean()
    if abs(post_mean - pre_mean) > threshold * uv_data['UV_1_280_mAU'].std():
        return True
    return False

sustained_changes = []
for i in range(min_duration, len(uv_data) - min_duration):
    if is_sustained_step_change(i):
        sustained_changes.append(i)

# Limit to max 10 regions
if len(sustained_changes) > 10:
    sustained_changes = sustained_changes[:10]

# Create plot
fig, ax = plt.subplots(figsize=(14, 7))
ax.plot(uv_data['Fraction_number'], uv_data['UV_1_280_mAU'], label='UV_1_280_mAU',
    alpha=0.7)

# Plot vertical lines at transitions
for t in transitions:
    ax.axvline(x=uv_data.iloc[t]['Fraction_number'], color='red', linestyle='--',
        linewidth=1)

# Highlight sustained step-change regions
for idx in sustained_changes:
    start = max(0, idx - 10)
    end = min(len(uv_data), idx + 10)
    ax.axhspan(uv_data.iloc[start]['Fraction_number'], uv_data.iloc[end]['
        Fraction_number'], alpha=0.2, color='green')

# Inset box plot of Z-scores
z_scores = (uv_data['UV_1_280_mAU'] - uv_data['UV_1_280_mAU'].mean()) / uv_data['
    UV_1_280_mAU'].std()
pre_transitions = z_scores.iloc[:transitions[0]] if transitions else z_scores
post_transitions = z_scores.iloc[transitions[0]:] if transitions else z_scores

ax_inset = fig.add_axes([0.65, 0.85, 0.25, 0.25])
ax_inset.boxplot([pre_transitions, post_transitions], patch_artist=True)
ax_inset.set_title('Z-scores Pre/Post Transition')
ax_inset.set_xlabel('Group')
ax_inset.set_ylabel('Z-score')

```

```
# Set labels and title
ax.set_title('Run_Boundary_Sensitivity')
ax.set_xlabel('Fraction_number')
ax.set_ylabel('UV_1_280_mAU')
ax.legend()
ax.grid(True, alpha=0.3)

# Save plot
plt.tight_layout()
plt.savefig('/workspace/run_boundary_analysis.png', dpi=300, bbox_inches='tight')
plt.close()
```

Listing 3: Code_3